AWS システム設計資料 - IoTセンサーデータ管理システム

システム概要

IoT Core でセンサーデバイスを接続し、時系列データを収集・処理・分析するシステム。リアルタイムのデータ処理と長期保存により、予測分析やダッシュボード作成を実現します。

対象用途:

- ビルディング管理（温度・湿度センサー）
- 産業設備の遠隔監視
- スマートホーム統合
- 環境モニタリング
- 製造ラインセンサー

アーキテクチャ構成

```
センサーデバイス群
├─ 温度センサー
├─ 湿度センサー
├─ 圧力センサー
└─ 動きセンサー
    ↓ MQTT 接続 (TLS)
AWS IoT Core
├─ Device Gateway (エンドポイント)
├─ Message Broker (MQTT)
├─ Device Shadow (デバイス状態)
└─ Fleet Provisioning
    ↓
IoT Rules (条件分岐・ルーティング)
    ├─ リアルタイム処理
    ├─ ルール評価
    └─ アクション実行
    ↓ フィルタリング・変換
TimeStream
├─ 時系列データベース
├─ ホットストレージ（記憶階層 ）
└─ コールドストレージ
    ↓
S3
└─ 長期アーカイブ

    ↓
Lambda
└─ カスタム処理

    ↓
CloudWatch
```

```
  ├─ メトリクス
  ├─ ダッシュボード
  └─ アラーム

      ↓
QuickSight
└─ ビジュアライゼーション・ダッシュボード
```

使用するAWSサービス

サービス	役割	責務内容
IoT Core	デバイス接続	MQTT ブローカー、デバイス認証
TimeStream	時系列DB	高スループットデータ保存
S3	ストレージ	長期データ保存、アーカイブ
Lambda	データ処理	リアルタイム処理、変換
QuickSight	ダッシュボード	データ可視化
IAM	アクセス制御	デバイス認証、権限管理

詳細設定値

AWS IoT Core 設定

```
エンドポイント: xxxxxxxxxxxx-ats.iot.ap-northeast-1.amazonaws.com:8883

Device Gateway (接続エンドポイント):
  - プロトコル: MQTT 3.1.1
  - ポート: 8883 (TLS 1.2/1.3)
  - 接続タイムアウト: 60秒
  - キープアライブ間隔: 60秒

メッセージブローカー設定:
  - 最大メッセージサイズ: 128 KB
  - QoS サポート: 0, 1
  - 保持メッセージ: 有効
  - セッション有効期限: 1時間

Device Shadow 設定:
  - 有効: ○
  - Shadow ドキュメント サイズ上限: 8 KB
  - Shadow バージョン追跡: 有効

Fleet Provisioning (デバイスプロビジョニング):
  - テンプレート名: SensorDeviceTemplate
  - クレーム証明書: X.509
  - 有効期限: 1日
```

```
接続認証:
  - 認証方式: X.509 証明書ベース
  - 証明書: デバイスあたり1つ
  - CA 証明書: 主要な CA 信頼済み
  - 認証フロー: Custom MQTT Authorizer 使用可

カスタム MQTT オーライザー:
  - 有効: ○
  - Lambda 関数: iot-auth-handler
  - キャッシュ: 5分
```

Thing (デバイス) 定義

```
Thing テンプレート例: sensor-device-template

Thing タイプ: SensorDevice
  - 説明: 温度・湿度センサー
  - 属性:
    * location: センサー設置場所
    * deviceModel: デバイスモデル
    * firmwareVersion: ファームウェアバージョン
    * group: 管理グループ

Thing 例: sensor-building-a-floor-2-room-201

Thing グループ:
  - ルートグループ: AllSensors
    * 子グループ: BuildingA
      - 子グループ: Floor2
        * デバイス: sensor-building-a-floor-2-room-201
        * デバイス: sensor-building-a-floor-2-room-202
      - 子グループ: Floor3

証明書設定:
  - 証明書 ID: CERTIFICATE_ID
  - 状態: ACTIVE
  - 有効期限: 2025年11月01日
  - 署名アルゴリズム: SHA256withRSA
  - Key Length: 2048 ビット

ポリシー設定:
  - ポリシー名: SensorPolicy

  ポリシードキュメント:
  {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": "iot:Connect",
        "Resource": "arn:aws:iot:ap-northeast-
```

```
1:ACCOUNT:client/${iot:Connection.Thing.ThingName}"
      },
      {
        "Effect": "Allow",
        "Action": "iot:Publish",
        "Resource": [
          "arn:aws:iot:ap-northeast-
1:ACCOUNT:topicfilter/sensors/${iot:Connection.Thing.ThingName}/telemetry",
          "arn:aws:iot:ap-northeast-
1:ACCOUNT:topicfilter/sensors/${iot:Connection.Thing.ThingName}/status"
        ]
      },
      {
        "Effect": "Allow",
        "Action": "iot:Subscribe",
        "Resource": "arn:aws:iot:ap-northeast-
1:ACCOUNT:topicfilter/sensors/${iot:Connection.Thing.ThingName}/commands"
      },
      {
        "Effect": "Allow",
        "Action": "iot:Receive",
        "Resource": "arn:aws:iot:ap-northeast-
1:ACCOUNT:topic/sensors/${iot:Connection.Thing.ThingName}/commands"
      }
    ]
  }
```

IoT Rules 設定

```
ルール1: PublishToTimeStream

ルール名: PublishToTimeStream
トピックフィルター: sensors/+/telemetry
クエリステートメント:
  SELECT
    clientId as deviceId,
    temperature,
    humidity,
    pressure,
    timestamp_ms() as timestamp
  FROM 'sensors/+/telemetry'

ルールペイロード:
  {
    "deviceId": "sensor-building-a-floor-2-room-201",
    "temperature": 23.5,
    "humidity": 65,
    "pressure": 1013,
    "timestamp": 1699008600000
  }
```

```
アクション:
  - タイプ: TimeStream
  - データベース: SensorMetrics
  - テーブル: DeviceReadings
  - ディメンションマッピング:
    * 次元1: deviceId (dimension)
    * 次元2: location (dimension)
  - メジャーカラム: temperature
  - メジャー値タイプ: DOUBLE
  - タイムスタンプ: timestamp
  - タイムスタンプ単位: MILLISECONDS

エラーアクション:
  - S3 への保存
  - バケット: iot-failed-messages
  - キープレフィックス: errors/
  - ロールARN: arn:aws:iam::ACCOUNT:role/iot-rules-role

---

ルール2: HighTemperatureAlert

ルール名: HighTemperatureAlert
トピックフィルター: sensors/+/telemetry
クエリステートメント:
  SELECT
    clientId as deviceId,
    temperature,
    humidity,
    timestamp_ms() as timestamp
  FROM 'sensors/+/telemetry'
  WHERE temperature > 30

アクション:
  - タイプ: SNS
  - ターゲット: arn:aws:sns:ap-northeast-1:ACCOUNT:iot-alerts
  - メッセージ形式:
    JSON:
      {
        "alert": "HIGH_TEMPERATURE",
        "deviceId": "${clientId}",
        "temperature": "${temperature}",
        "timestamp": "${timestamp_ms()}",
        "action": "CHECK_DEVICE_IMMEDIATELY"
      }

---

ルール3: AnomalyDetection

ルール名: AnomalyDetection
トピックフィルター: sensors/+/telemetry
クエリステートメント:
  SELECT
```

```
    clientId,
    temperature,
    humidity,
    timestamp_ms() as timestamp
  FROM 'sensors/+/telemetry'
  WHERE
    temperature > 35 OR
    temperature < 10 OR
    humidity > 95 OR
    humidity < 5

アクション1: Lambda
  - 関数名: anomaly-handler
  - 非同期呼び出し

アクション2: S3
  - バケット: iot-anomalies
  - キーペアテンプレート: anomalies/${year}/${month}/${day}/${hour}/${clientId}.json
  - コンテンツフォーマット: JSON

---

ルール4: PublishToS3Archive

ルール名: PublishToS3Archive
トピックフィルター: sensors/+/telemetry
クエリステートメント:
  SELECT *
  FROM 'sensors/+/telemetry'

アクション:
  - タイプ: S3
  - バケット: iot-archive
  - キープレフィックス: raw-data/${year}/${month}/${day}/${hour}/
  - キーペアテンプレート: ${clientId}-${timestamp_ms()}.json
  - MIME タイプ: application/json
  - コンテンツフォーマット: JSON
  - タイムスタンプ単位: SECONDS
```

TimeStream テーブル設定

```
データベース名: SensorMetrics

テーブル名: DeviceReadings

テーブル設定:
  - 保持ポリシー（記憶階層）:
    * ホットストレージ: 1時間（最新データ、高速アクセス）
    * コールドストレージ: 364日（アーカイブ用、低コスト）

ディメンション定義:
```

```
  1. ディメンション名: deviceId
     - タイプ: VARCHAR
     - 説明: センサーデバイスID

  2. ディメンション名: location
     - タイプ: VARCHAR
     - 説明: センサー設置位置

  3. ディメンション名: sensorType
     - タイプ: VARCHAR
     - 説明: センサー種類（temperature, humidity など）

  4. ディメンション名: unit
     - タイプ: VARCHAR
     - 説明: 測定単位（°C, % など）

メジャー定義:
  1. メジャー名: temperature
     - メジャー値タイプ: DOUBLE
     - 単位: °C

  2. メジャー名: humidity
     - メジャー値タイプ: DOUBLE
     - 単位: %

  3. メジャー名: pressure
     - メジャー値タイプ: DOUBLE
     - 単位: hPa

データ挿入:
  - 単位: ms
  - 最小レコード時間: 2024年1月1日（過去データ許容）
  - 最大レコード時間: 現在時刻 + 15分（未来データ許容）

インデックス設定:
  - プライマリキー: deviceId, sensorType
  - パーティションキー最適化: 有効

スケーリング:
  - モード: オンデマンド（自動スケーリング）
  - スループット: 自動調整
```

Lambda 関数設定（異常検知）

```
関数名: anomaly-handler

実行時: Python 3.11

メモリ: 512 MB
タイムアウト: 60秒
```

```
環境変数:
  - TIMESTREAM_DB: SensorMetrics
  - TIMESTREAM_TABLE: DeviceReadings
  - SNS_TOPIC_ARN: arn:aws:sns:...
  - ANOMALY_THRESHOLD: 3  # 標準偏差の倍数

コード例：

```python
import boto3
import json
from datetime import datetime, timedelta
import statistics

timestream = boto3.client('timestream-query')
sns = boto3.client('sns')

def lambda_handler(event, context):
    """
    異常検知処理
    """
    try:
        record = json.loads(event['body']) if 'body' in event else event

        device_id = record.get('deviceId')
        temperature = record.get('temperature')
        humidity = record.get('humidity')
        timestamp = record.get('timestamp')

        # 過去24時間の履歴取得
        history = get_device_history(device_id, 24)

        if len(history) < 10:
            print(f'Not enough historical data for {device_id}')
            return

        # 統計分析
        temps = [h['temperature'] for h in history]
        mean_temp = statistics.mean(temps)
        stdev_temp = statistics.stdev(temps)

        # Z スコア計算
        z_score = abs((temperature - mean_temp) / stdev_temp) if stdev_temp > 0
else 0

        # 異常判定
        threshold = float(os.environ.get('ANOMALY_THRESHOLD', 3))

        if z_score > threshold:
            send_alert(device_id, temperature, mean_temp, z_score)

        return {'statusCode': 200, 'message': 'Processed'}

    except Exception as e:
```
```

```
        print(f'Error: {str(e)}')
        return {'statusCode': 500, 'error': str(e)}

def get_device_history(device_id, hours):
    """過去データ取得"""
    query = f"""
    SELECT temperature, humidity, time
    FROM {os.environ['TIMESTREAM_TABLE']}
    WHERE deviceId = '{device_id}'
    AND time > ago({hours}h)
    ORDER BY time DESC
    """

    try:
        response = timestream.query(QueryString=query)

        rows = []
        for row in response['Rows']:
            rows.append({
                'temperature': float(row['Data'][0]['ScalarValue']),
                'humidity': float(row['Data'][1]['ScalarValue']),
                'time': row['Data'][2]['ScalarValue']
            })

        return rows

    except Exception as e:
        print(f'Query error: {str(e)}')
        return []

def send_alert(device_id, current_temp, avg_temp, z_score):
    """アラート送信"""
    message = f"""
    異常検知アラート

    デバイスID: {device_id}
    現在温度: {current_temp}°C
    平均温度: {avg_temp:.1f}°C
    異常度 (Z スコア): {z_score:.2f}

    確認してください。
    """

    sns.publish(
        TopicArn=os.environ['SNS_TOPIC_ARN'],
        Subject='IoT 異常検知アラート',
        Message=message
    )
```

QuickSight ダッシュボード設定

```
ダッシュボード名: SensorMetrics Dashboard

データソース: Amazon TimeStream (SensorMetrics.DeviceReadings)

ビジュアル1: リアルタイム温度モニター
  - タイプ: KPI
  - フィールド: temperature (avg)
  - フィルター: 直近1分
  - ゲージ範囲: 0 - 50°C

ビジュアル2: 温度時系列グラフ
  - タイプ: 折れ線グラフ
  - X軸: time (分単位)
  - Y軸: temperature (平均)
  - 凡例: deviceId
  - 期間: 過去24時間

ビジュアル3: 湿度分布
  - タイプ: 散布図
  - X軸: deviceId
  - Y軸: humidity (average)
  - サイズ: temperature
  - 期間: 過去24時間

ビジュアル4: デバイス別アラート
  - タイプ: テーブル
  - カラム: deviceId, temperature, humidity, status
  - フィルター: temperature > 30 OR humidity > 90
  - ソート: temperature (降順)

ビジュアル5: 異常イベント
  - タイプ: 棒グラフ
  - X軸: time (日単位)
  - Y軸: count
  - フィルター: status = "anomaly"

ビジュアル6: 地理的ヒートマップ
  - タイプ: Geospatial
  - ロケーション: location
  - カラー: temperature
  - バリュー: avg(temperature)
```

データフロー例

デバイスからのデータ送信

```
デバイス側コード (Python / MicroPython):

```python
import paho.mqtt.client as mqtt
```
```

```
import json
import time
import random
from datetime import datetime

# 接続パラメータ
ENDPOINT = "xxxxxxxxxxxx-ats.iot.ap-northeast-1.amazonaws.com"
PORT = 8883
CLIENT_ID = "sensor-building-a-floor-2-room-201"
CERT_FILE = "device.pem.crt"
KEY_FILE = "private.pem.key"
CA_FILE = "AmazonRootCA1.pem"

# MQTT クライアント設定
client = mqtt.Client(client_id=CLIENT_ID)
client.tls_set(cafile=CA_FILE, certfile=CERT_FILE, keyfile=KEY_FILE,
cert_reqs=mqtt.ssl.CERT_REQUIRED, tls_version=mqtt.ssl.PROTOCOL_TLSv1_2)
client.tls_insecure_set(False)

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print(f"Connected to IoT Core: {CLIENT_ID}")
    else:
        print(f"Failed to connect, return code {rc}")

def on_disconnect(client, userdata, rc):
    print(f"Disconnected, return code {rc}")

client.on_connect = on_connect
client.on_disconnect = on_disconnect

# 接続
client.connect(ENDPOINT, PORT, keepalive=60)
client.loop_start()

# センサーデータ送信ループ
while True:
    # センサー値取得（シミュレーション）
    temperature = 20 + random.uniform(-2, 5)
    humidity = 50 + random.uniform(-10, 20)
    pressure = 1013 + random.uniform(-5, 5)

    # ペイロード作成
    payload = {
        "temperature": round(temperature, 2),
        "humidity": round(humidity, 2),
        "pressure": round(pressure, 2),
        "timestamp": int(datetime.utcnow().timestamp() * 1000)
    }

    # パブリッシュ
    topic = f"sensors/{CLIENT_ID}/telemetry"
    client.publish(topic, json.dumps(payload), qos=1)
```

```
        print(f"Published: {payload}")

        # 60秒ごと送信
        time.sleep(60)

    client.loop_stop()
    client.disconnect()
```

推定月額コスト

シナリオ: 1000デバイス、60秒ごと送信、30日間

項目	計算	月額費用
IoT Core		
デバイス接続	1000 × 無料枠	$0.00
メッセージング	1000 × (60秒/60) × 24h × 30d = 43.2M	$21.60
TimeStream		
データ取得	43.2M レコード	$21.60
ホットストレージ	1GB/日 × 1日	$0.53
コールドストレージ	500GB (30日-1日)	$30.00
S3		
ストレージ	500GB	$11.50
PUT リクエスト	1000万	$50.00
GET リクエスト	1000万	$4.00
Lambda		
異常検知実行	100万実行	$0.20
実行時間	5,000,000 GB-秒	$83.33
QuickSight		
SPICE (5GBまで無料)	無料枠内	$0.00
ユーザー (最初の1人)	1ユーザー	$24.00
その他		
SNS 通知	1000通知	$0.50
CloudWatch	ログ・メトリクス	$15.00
合計		**$262.26**

セキュリティ設定

デバイス認証フロー

```
1. 証明書の生成・配置
   ├─ デバイス証明書 (device.pem.crt)
   ├─ 秘密キー (private.pem.key)
   └─ CA 証明書 (AmazonRootCA1.pem)

2. TLS ハンドシェイク
   ├─ デバイス → IoT Core: ClientHello (証明書付き)
   ├─ IoT Core: 証明書検証
   └─ 確立: TLS 1.2 / 1.3 セッション

3. MQTT コネクション
   └─ ClientID: Thing名
       ユーザー名: (空)
       パスワード: (不要)

4. トピック権限確認
   ├─ ポリシー評価
   └─ パブリッシュ/サブスクリプト許可確認
```

IoT ポリシー例 (セキュアな設定)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "iot:Connect",
      "Resource": [
        "arn:aws:iot:ap-northeast-1:ACCOUNT:client/sensor-*"
      ],
      "Condition": {
        "IpAddress": {
          "iot:Connection.IpAddress": [
            "203.0.113.0/24"
          ]
        }
      }
    },
    {
      "Effect": "Allow",
      "Action": "iot:Publish",
      "Resource": [
        "arn:aws:iot:ap-northeast-
1:ACCOUNT:topicfilter/sensors/${iot:Connection.Thing.ThingName}/telemetry"
      ]
    },
    {
      "Effect": "Allow",
```

```
      "Action": "iot:Subscribe",
      "Resource": [
        "arn:aws:iot:ap-northeast-
1:ACCOUNT:topicfilter/sensors/${iot:Connection.Thing.ThingName}/commands"
      ]
    },
    {
      "Effect": "Deny",
      "Action": "iot:*",
      "Resource": "*",
      "Condition": {
        "IpAddress": {
          "iot:Connection.IpAddress": "203.0.113.99"
        }
      }
    }
  ]
}
```

監視とログ

CloudWatch メトリクス

```
監視対象:
- IoT Core:
  * Connect.Success: 接続成功
  * Connect.Failure: 接続失敗
  * Publish.In.Success: パブリッシュ成功
  * Subscribe.Success: サブスクリプション成功

- TimeStream:
  * UserErrors: ユーザーエラー
  * SystemErrors: システムエラー
  * SuccessfulRequests: 成功リクエスト

- Lambda:
  * Invocations: 実行回数
  * Duration: 実行時間
  * Errors: エラー数
```

デバイス管理のベストプラクティス

1. デバイス登録フロー

```
新規デバイス → Fleet Provisioning → 証明書発行 → テスト接続 → 本番デプロイ
```

2. 定期的なセキュリティ更新

```
- 証明書有効期限: 1年
- 更新スケジュール: 期限90日前
- ローテーション: ブルーグリーンデプロイメント
```

3. デバイスグループの活用

```
- 組織単位でグループ化
- グループレベルのポリシー適用
- 一括更新・再起動
```